

Politechnika Warszawska
Wydział Matematyki i Nauk Informacyjnych

Bamboo Garden Trimming

Przygotowywanie ogrodu bambusowego –
algorytmiczne aspekty problemu harmonogramowania wieczystego

Projekt zaliczeniowy z przedmiotu
Gry Kombinatoryczne

Autorzy: Michał Bober
Piotr Komisarczyk
Igor Staręga

Data: 5 czerwca 2026

Praca oparta na artykule:
D. Bilò, L. Gualà, S. Leucci, G. Proietti, G. Scornavacca,
Cutting bamboo down to size,
Theoretical Computer Science 909 (2022), ss. 54–67.

Streszczenie

Niniejszy raport omawia problem *Bamboo Garden Trimming* (BGT) – harmonogramowania wieczystego sekwencji cięć robota-ogrodnika dbającego o ogród bambusowy. Zadanie polega na wyznaczeniu nieskończonego harmonogramu cięć minimalizującego *makespan*, czyli największą wysokość kiedykolwiek osiągniętą przez dowolny bambus. Przedstawiamy formalny model BGT, intuicję stojącą za redukcją do problemu Pinwheel Scheduling oraz dwie naturalne strategie: Reduce-Max (przycinaj aktualnie najwyższy bambus) i Reduce-Fastest(x) (przycinaj najszybciej rosnący bambus spośród tych, które osiągnęły próg x). Omawiamy najważniejsze wyniki z pracy Bilò i wsp.: pierwsze stałe ograniczenie górne 9 dla Reduce-Max oraz ograniczenie $\frac{3+\sqrt{5}}{2} < 2,62$ dla Reduce-Fastest(x) przy $x = 1 + \frac{1}{\sqrt{5}}$. Ostatnia część raportu opisuje naszą implementację w Pythonie i Streamlit: interaktywną aplikację pozwalającą symulować, porównywać i testować omawiane strategie.

Spis treści

1	Wprowadzenie i motywacja	3
1.1	Znaczenie praktyczne	3
2	Formalna definicja problemu	3
2.1	Normalizacja i założenia	4
2.2	Ograniczenia na makespan	4
3	Znane strategie heurystyczne	5
3.1	Strategia Reduce-Max	5
3.1.1	Szkic dowodu	5
3.2	Strategia Reduce-Fastest(x)	6
3.2.1	Kluczowe idee dowodu	7
3.3	Porównanie heurystyk	7
4	Powiązanie z problemem Pinwheel Scheduling	7
4.1	Redukcja BGT $\rightarrow$ PS dla makespanu 2	8
5	Trimming Oracles – struktury danych	8
5.1	Oracle dla Reduce-Fastest(x)	8
5.2	Oracle dla Reduce-Max	9
5.3	Oracle gwarantujący makespan 2	9
6	Implementacja: interaktywna aplikacja webowa	10
6.1	Architektura	10
6.2	Tryby działania	11
6.3	Kluczowe decyzje implementacyjne	11
6.4	Uruchomienie i testy	11
6.5	Wkład własny	12
7	Analiza złożoności i podsumowanie wyników	12
7.1	Zestawienie złożoności Oracle’i	12
7.2	Otwarte problemy	13
8	Wnioski	13

1. Wprowadzenie i motywacja

Problem *Bamboo Garden Trimming* (BGT) jest matematyczną formalizacją pozornie banalnego zadania utrzymania ogrodu bambusowego. Wyobraźmy sobie dom nad jeziorem, przed którym rośnie ogród złożony z n bambusów. Każdy bambus b_i rośnie o stałą wartość $h_i > 0$ na dobę. Robot-ogrodnik może raz dziennie skosić *co najwyżej jeden* bambus, natychmiast resetując jego wysokość do zera. Pytanie brzmi: jak zaplanować sekwencję cięć, aby widok na jezioro był jak najmniej zasłonięty?

Problem ten, sformalizowany przez Gąsieńca i współautorów w [2], łączy w sobie zagadnienia z teorii szeregowania zadań, gier kombinatorycznych oraz struktury danych. Pomimo prostego opisu, prowadzi do interesujących pytań dotyczących granicy między scenariuszem statycznym a adversarialnym.

1.1. Znaczenie praktyczne

Choć historia z panda-ogrodnikiem jest celowo lekka, model BGT opisuje realny typ problemów: ograniczony zasób obsługi musi cyklicznie reagować na obiekty narastające z różnymi szybkościami. Problem BGT i jego uogólnienia pojawiają się między innymi w następujących zastosowaniach:

- **Zarządzanie buforami sieciowymi** – pakiety o różnych priorytetach wymagają cyklicznego przetwarzania [8];
- **Monitorowanie usług w chmurze** – maszyny wirtualne wymagają okresowej inspekcji pod kątem anomalii [6];
- **Harmonogramowanie czasu rzeczywistego** – zadania z różnymi częstotliwościami obsługi muszą być realizowane bez przekroczeń terminów [7];
- **Problem kubka** (*cup game*) – uogólnienie BGT, w którym dzienne cięcie może przekroczyć 1 jednostkę.

2. Formalna definicja problemu

Definicja 2.1 (Problem BGT). Ogród zawiera n bambusów $b_1, \dots, b_n$, gdzie bambus b_i ma dzienny współczynnik wzrostu $h_i > 0$. Zakładamy, że

$$\sum_{i=1}^n h_i = 1$$

(bez straty ogólności – skalowanie nie zmienia struktury optymalnego harmonogramu). Początkowo wszystkie bambusy mają wysokość 0. Na końcu każdego dnia $d \geq 1$ ogrodnik może przyciąć *co najwyżej jeden* bambus, natychmiast resetując jego wysokość do 0.

Wysokość bambusu b_i na końcu dnia d (przed decyzją o cięciu) wynosi:

$$H_i(d) = (d - d') \cdot h_i,$$

gdzie $d' < d$ to ostatni dzień, w którym b_i był cięty (przyjmujemy $d' = 0$, jeśli b_i nigdy nie był cięty).

Celem jest wyznaczenie *wieczystego harmonogramu* (nieskończonej sekwencji decyzji o cięciu) minimalizującego *makespan*:

$$\mathcal{M} = \max_{i,d} H_i(d).$$

2.1. Normalizacja i założenia

Warunek $\sum h_i = 1$ jest uzasadniony następująco: jeśli oryginalne współczynniki sumują się do $S \neq 1$, wystarczy podzielić każde h_i przez S . Makespan skaluje się proporcjonalnie, a struktura optymalnego harmonogramu pozostaje niezmienną.

Przyjmujemy standardowe uporządkowanie $h_1 \geq h_2 \geq \dots \geq h_n > 0$.

2.2. Ograniczenia na makespan

Poniższe twierdzenie ustala dolne i górne granice optymalnego makespanu.

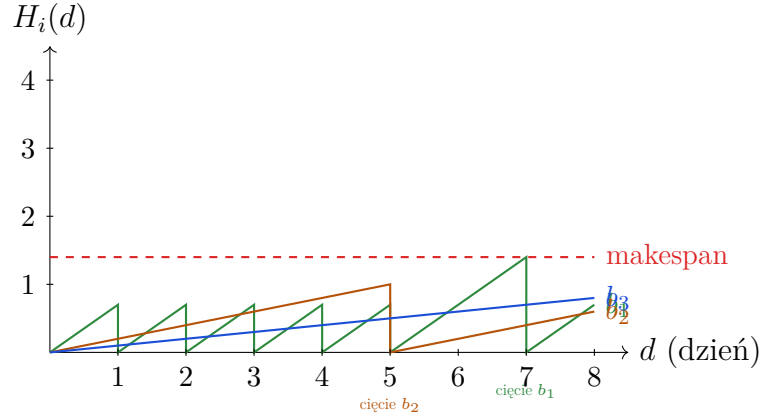
Twierdzenie 2.2 (Ograniczenia na makespan). *Dla każdej instancji BGT:*

- (i) $\mathcal{M} \geq 1$ dla każdego harmonogramu;
- (ii) istnieje instancja, dla której $\mathcal{M}$ może być dowolnie bliski 2;
- (iii) makespan 2 jest osiągalny dla każdej instancji.

Dowód. (i) W każdym dniu suma wysokości wszystkich bambusów rośnie o 1. Jeżeli makespan byłby mniejszy niż 1, każde pojedyncze cięcie usuwałoby mniej niż 1 jednostkę wysokości. W długim horyzoncie średnia wysokość usuwana przez ogrodnika byłaby więc mniejsza niż dzienny przyrost, co prowadziłoby do nieograniczonego wzrostu łącznej wysokości. Sprzeczność.

(ii) Rozważmy dwa bambusy: $h_1 = 1 - \varepsilon$ oraz $h_2 = \varepsilon$. Aby bambus b_2 nie rósł bez ograniczeń, musi zostać od czasu do czasu przycięty. W dniu, w którym ogrodnik tnie b_2 , nie może jednocześnie przyciąć b_1 . Nawet jeśli b_1 był cięty dzień wcześniej, to przed kolejną możliwą interwencją urośnie przez dwa dni, osiągając wysokość $2(1 - \varepsilon) = 2 - 2\varepsilon$. Dla $\varepsilon \rightarrow 0$ otrzymujemy rodzinę instancji wymagających makespanu dowolnie bliskiego 2.

(iii) Osiągalność makespanu 2 wynika z redukcji do problemu Pinwheel Scheduling opisaney w sekcji 4. $\square$



Rysunek 1: Przykład ewolucji wysokości trzech bambusów ($h_1 = 0,7$, $h_2 = 0,2$, $h_3 = 0,1$) według strategii Reduce-Max. Czerwona linia przerywana oznacza osiągnięty makespan.

3. Znane strategie heurystyczne

3.1. Strategia Reduce-Max

Definicja 3.1 (Reduce-Max). W każdym dniu ogrodnik przycina bambus o *największej aktualnej wysokości*. Remisy rozstrzygane są dowolnie.

Reduce-Max jest najbardziej naturalną heurystyką i zachowuje się bardzo dobrze w praktyce. Wyniki eksperymentalne sugerują, że jej makespan może być ograniczony przez 2 [5], jednak najlepsze znane ograniczenie górne w scenariuszu adwersarialnym wynosi $1 + \mathcal{H}_{n-1} = \Theta(\log n)$, gdzie $\mathcal{H}_{n-1}$ to $(n-1)$ -ta liczba harmoniczna [4].

Kluczowym wynikiem artykułu Bilò i wsp. jest:

Twierdzenie 3.2 ([1], Twierdzenie 1). Reduce-Max *gwarantuje makespan co najwyżej 9*.

Jest to **pierwsze stałe ograniczenie górne** dla tej strategii w scenariuszu nieadwersarialnym, wykazując rozdział między scenariuszem statycznym a adwersarialnym.

3.1.1 Szkic dowodu

Podzielmy bambusy na *poziomy*: bambus b_i jest na poziomie $j \geq 1$, jeśli

$$\frac{1}{2^j} \leq h_i < \frac{1}{2^{j-1}}.$$

Niech L_j oznacza zbiór bambusów poziomu j , a $\sigma(j)$ – maksymalną wysokość kiedykolwiek osiągniętą przez dowolny bambus poziomu $\geq j$ (z $\sigma(K+1) = 0$). Kluczowa rekurencja to:

$$\sigma(j) \leq \max\{3, \sigma(j+1)\} + 3 \sum_{k=1}^j \frac{|L_k|}{2^j}. \quad (1)$$

Argument o powtarzanych cięciach. Rozważmy okno $[d_0+1, d_1]$, w którym Reduce-Max tnie wyłącznie bambusy poziomu $\leq j$ o wysokości ≥ 3 . Cięcia dzielimy na:

- *pierwsze cięcia* – bambus cięty po raz pierwszy w oknie; jest ich co najwyżej $\sum_{k=1}^j |L_k|$;
- *powtarzane cięcia* – bambus cięty ponownie; każde usuwa co najmniej 3 jednostki wzrostu, więc stanowią co najwyżej $\frac{1}{3}$ wszystkich dni okna.

Stąd $d_1 - d_0 \leq \frac{3}{2} \sum_{k=1}^j |L_k|$.

Sumując wkłady bambusów i korzystając z $\sum h_i = 1$:

$$\sigma(1) \leq 3 + 3 \sum_{j=1}^K \sum_{k=1}^j \frac{|L_k|}{2^j} \leq 3 + 6 \sum_{i=1}^n h_i = 9. \quad \square$$

3.2. Strategia Reduce-Fastest(x)

Definicja 3.3 (Reduce-Fastest(x)). W każdym dniu ogrodnik przycina bambus o *najwyższym współczynniku wzrostu* spośród tych, które osiągnęły wysokość co najmniej x . Jeśli żaden bambus nie przekroczył progu x , strategia może nie wykonać cięcia; w naszej implementacji stosujemy praktyczny wariant z awaryjnym wyborem Reduce-Max, żeby symulacja zawsze wykonywała jedno sensowne działanie dziennie.

Twierdzenie 3.4 ([1], Twierdzenie 2). *Dla stałego $x > 1$, makespan Reduce-Fastest(x) jest ograniczony przez:*

$$\mathcal{M} \leq \max \left\{ x + \frac{x^2}{4(x-1)}, \quad \frac{1}{2} + x + \frac{x^2}{4(x-\frac{1}{2})} \right\}.$$

Wniosek 3.5. *Dla $x = 1 + \frac{1}{\sqrt{5}}$ (optymalny wybór) makespan Reduce-Fastest(x) wynosi co najwyżej $1 + \varphi = \frac{3+\sqrt{5}}{2} \approx 2,618$, gdzie $\varphi = \frac{1+\sqrt{5}}{2}$ to złota liczba.*

Dla $x = 2$ (poprzednio uważany za najlepszy) uzyskujemy poprawione ograniczenie $\frac{19}{6} \approx 3,17$, lepsze od wcześniejszego wyniku 4.

3.2.1 Kluczowe idee dowodu

Niech $[0, T]$ oznacza interwał, w którym bambus b_i nie jest cięty (od momentu gdy osiągnął wysokość $\geq x$). Definiujemy:

- *objętość* $V = T$ – całkowity wzrost ogrodu w $[0, T - 1]$;
- N – liczba różnych bambusów ciętych w $[0, T - 1]$;
- V' – objętość usunięta przez powtarzane cięcia.

Ponieważ każde powtarzane cięcie usuwa co najmniej x jednostek: $V' \leq T(1 - h_i) - N^2 h_i$. Z warunkiem $h_j \geq h_i$ dla każdego ciętego b_j , wyprowadza się:

$$T \leq \frac{x^2}{4h_i(h_i + x - 1)}.$$

Makespan wynosi $M \leq x + h_i + Th_i$, co po optymalizacji po h_i daje wzór z twierdzenia 3.4.

3.3. Porównanie heurystyk

Tabela 1: Zestawienie znanych granic makespanu dla omawianych strategii.

Strategia	Ograniczenie górne	Ograniczenie dolne	Złożoność oracle
Reduce-Max (adwersarialny)	$\Theta(\log n)$, ciasne	$\Theta(\log n)$	$O(\log^2 n)$ lub $O(\log n)$ amortyzowane
Reduce-Max (statyczny)	≤ 9 [nowy]	≥ 1	$O(\log^2 n)$
Reduce-Fastest($x = 2$)	$\leq 19/6$ [poprawa]	brak znanych	$O(\log n)$
Reduce-Fastest($x = 1 + 1/\sqrt{5}$)	$\leq (3 + \sqrt{5})/2 \approx 2,62$ [nowy]	brak znanych	$O(\log n)$
Optymalny (Pinwheel)	≤ 2 , asymptotycznie ciasne	$\geq 2 - \varepsilon$	$O(\log n)$

4. Powiązanie z problemem Pinwheel Scheduling

Definicja 4.1 (Pinwheel Scheduling, PS). Dane jest n zadań z całkowitymi częstotliwościami $f_i \geq 1$. Szukamy wieczystego harmonogramu S takiego, że każde zadanie i pojawia się co najmniej raz w każdym spójnym podciągu długości f_i . *Gęstość* instancji definiujemy jako $\rho = \sum_{i=1}^n \frac{1}{f_i}$.

Twierdzenie 4.2 (Warunek konieczny i dostateczny, [3]). *Instancja PS z $\rho > 1$ nie posiada dopuszczalnego harmonogramu. Jeśli $\rho \leq 1$ i wszystkie f_i są potęgami 2, dopuszczalny harmonogram zawsze istnieje.*

4.1. Redukcja BGT $\rightarrow$ PS dla makespanu 2

Aby uzyskać makespan ≤ 2 , chcemy ciąć bambus b_i co najwyżej $\lfloor 2/h_i \rfloor$ dni. Definiujemy:

$$f_i = 2^{\lfloor \log_2(2/h_i) \rfloor} \quad (\text{zaokrąglenie } \lfloor 2/h_i \rfloor \text{ w dół do potęgi } 2).$$

Gęstość wynikowej instancji PS:

$$\rho = \sum_{i=1}^n \frac{1}{f_i} \leq \sum_{i=1}^n h_i = 1.$$

Ponieważ f_i są potęgami 2, istnieje dopuszczalny harmonogram PS, który bezpośrednio daje harmonogram BGT z makespanem ≤ 2 .

Problem praktyczny. Wszystkie dopuszczalne harmonogramy instancji PS mogą mieć długość $\Omega(\prod_{i=1}^n f_i)$ [3], co czyni naiwne podejście wykładniczym. Rozwiązaniem jest *Trimming Oracle* (patrz sekcji 5).

Relacja między BGT i Pinwheel Scheduling wpisuje się w szerszy nurt badań nad harmonogramami okresowymi. Klasy dopuszczalnych instancji Pinwheel były rozszerzane między innymi w [12], natomiast dla dyskretnego BGT rozwijano także niezależne podejścia aproksymacyjne, np. [13].

5. Trimming Oracles – struktury danych

Definicja 5.1 (Trimming Oracle). Trimming Oracle to struktura danych utrzymująca stan ogrodu i odpowiadająca na zapytanie „Który bambus ciąć dzisiaj?” w czasie subliniowym, używając przestrzeni liniowej.

Poniżej opisujemy trzy wyrocznie (*oracles*) z artykułu [1].

5.1. Oracle dla Reduce-Fastest(x)

Idea. Dla każdego bambusa b_i utrzymujemy dzień d_i , w którym b_i osiągnie wysokość $\geq x$. Zapytanie zwraca b_i z minimalnym indeksem i spośród tych, dla których $d_i \leq D$.

Struktura danych. Używamy drzewa wyszukiwania priorytetowego (priority search tree, PST) [9] utrzymującego punkty (d_i, i) z operacjami:

- $\text{MinYInXRange}(T, x_0)$: minimum i wśród (d_i, i) z $d_i \leq x_0$;
- $\text{GetX}(T, y)$: współrzędna d_i punktu o $y = i$.

Obie operacje działają w czasie $O(\log n)$.

Problem rosnących indeksów. Dni d_i i D rosną bezgranicznie. Rozwiązanie: dzie-

limy czas na *interwały* $I_j = [nj, n(j+1) - 1]$ i utrzymujemy dwa drzewa T_1, T_2 dla bieżącego i następnego interwału, mierząc dni od początku interwału.

Algorytm 1 Trimming Oracle dla Reduce-Fastest(x) – funkcja Query

Wejście: dzień δ w bieżącym interwale I_j

Wyjście: indeks bambusa do przycięcia (lub brak akcji)

```

1:  $i \leftarrow \text{MinYInXRange}(T_1, \delta)$ 
2: if  $i$  istnieje then
3:   Update( $T_1, \delta + \lceil x/h_i \rceil, i$ )
4:   Update( $T_2, \delta + \lceil x/h_i \rceil - n, i$ )
5: end if
6: Zaktualizuj  $T_2$  dla bambusa  $b_{\delta+1}$ 
7:  $\delta \leftarrow (\delta + 1) \bmod n$ 
8: if  $\delta = 0$  then
9:   Zamień  $T_1$  i  $T_2$ 
10: end if
11: return „Przytnij  $b_i$ ” (lub „brak akcji”)

```

Twierdzenie 5.2 ([1], Twierdzenie 3). *Istnieje Trimming Oracle implementujący Reduce-Fastest(x) o przestrzeni $O(n)$, czasie budowy $O(n \log n)$ i czasie zapytania $O(\log n)$ w pesymistycznym przypadku.*

5.2. Oracle dla Reduce-Max

Idea. Każdy bambus b_i reprezentujemy prostą $\ell_i(d) = h_i d + c_i$ opisującą jego wysokość w dniu d . Zapytanie wymaga znalezienia $\arg \max_i \ell_i(D)$, co odpowiada obliczeniu *górnej otoczki* (upper envelope) zbioru prostych.

Struktura danych. Używamy dynamicznej struktury utrzymującej górną otoczkę [10, 11], obsługując:

- wstawianie i usuwanie prostych: $O(\log^2 n)$ pesymistycznie lub $O(\log n)$ amortyzowanym;
- zapytanie $\text{Upper}(U, d)$: prosta maksymalna w punkcie d w czasie $O(\log n)$.

Twierdzenie 5.3 ([1], Twierdzenie 4). *Istnieje Trimming Oracle implementujący Reduce-Max o przestrzeni $O(n)$, czasie budowy $O(n \log n)$, czasie zapytania $O(\log^2 n)$ pesymistycznie lub $O(\log n)$ amortyzowanym.*

5.3. Oracle gwarantujący makespan 2

Idea. Zaokrąglamy współczynniki h_i do najbliższych mniejszych potęg dwójki i konstruujemy wieczyste harmonogramy dla tak zwanych *instancji regularnych*.

Definicja 5.4 (Instancja regularna). Instancja jest *regularna*, jeśli bambus b_i ma współczynnik wzrostu $h_i = 2^{-i}$ dla $i = 1, \dots, n-1$ i $h_n = h_{n-1} = 2^{-(n-1)}$.

Dla instancji regularnych istnieje elegancka reguła: bambus do przycięcia w dniu D to b_i , gdzie i jest pozycją *najmniej znaczącego bitu równego 0* w binarnej reprezentacji D . Jeśli wszystkie $n - 1$ najmłodszych bitów wynosi 1, tniemy b_n .

Twierdzenie 5.5 ([1], Lemat 1). *Dla instancji regularnych istnieje Trimming Oracle o przestrzeni $O(n)$, czasie budowy $O(n)$ i czasie zapytania $O(1)$ pesymistycznie, gwarantujący makespan 1.*

Dla ogólnych instancji stosuje się hierarchiczną dekompozycję: bambusy są iteracyjnie łączone w *wirtualne bambusy*, tworząc drzewo T_O o logarytmicznej głębokości. Każdy węzeł wewnętrzny zawiera Oracle dla instancji regularnej (skalibrowanej lokalnie).

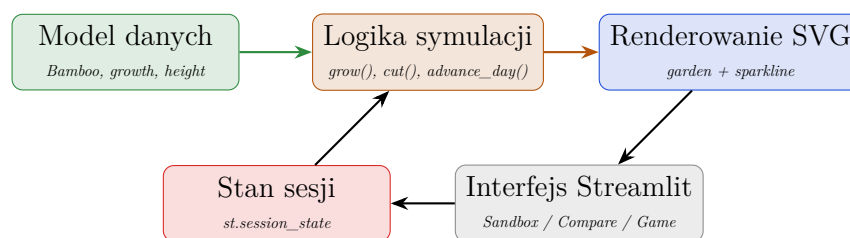
Twierdzenie 5.6 ([1], Twierdzenie 5). *Istnieje Trimming Oracle gwarantujący makespan 2, o przestrzeni $O(n)$, czasie budowy $O(n \log n)$ i czasie zapytania $O(\log n)$.*

6. Implementacja: interaktywna aplikacja webowa

W ramach projektu przygotowaliśmy interaktywną aplikację webową wizualizującą problem BGT. Aktualna wersja repozytorium jest napisana w **Pythonie** z użyciem biblioteki **Streamlit**. Taki wybór technologii dobrze pasuje do charakteru projektu: najważniejsza jest czytelna logika symulacji, szybkie eksperymentowanie z parametrami oraz proste uruchomienie aplikacji bez osobnego frontendu i backendu.

Aplikacja nie korzysta z zewnętrznych bibliotek obliczeniowych. Strategie Reduce-Max, Reduce-Fastest(x) oraz wariant oracle są zaimplementowane bezpośrednio w kodzie projektu, a wizualizacje ogrodu i wykresów generowane są jako wektorowe rysunki SVG po stronie Pythona.

6.1. Architektura



Rysunek 2: Architektura aktualnej aplikacji BGT. Część obliczeniowa jest wydzielona na czyste funkcje, a Streamlit odpowiada za stan sesji, kontrolki i prezentację.

Najważniejsze elementy kodu:

- **Bamboo** – niemutowalna struktura opisująca pojedynczy bambus;
- **grow()** i **cut()** – podstawowe operacje na ogrodzie;

- `advance_day()` – wspólna funkcja wykonująca jeden dzień symulacji dla dowolnej strategii;
- `oracle_reduce_max()`, `oracle_reduce_fastest()` oraz `oracle_optimal()` – implementacje reguł wyboru cięcia;
- `bamboo_garden_svg()` i `sparkline_svg()` – generatory wizualizacji w SVG.

6.2. Tryby działania

Aplikacja oferuje trzy tryby:

1. **Sandbox** – użytkownik konfiguruje ogród (liczba bambusów, współczynniki wzrostu), wybiera algorytm spośród Reduce-Max, Reduce-Fastest(x) (z regulowanym progiem x) i optymalnego Oracle, obserwuje animowaną symulację dzień po dniu.
2. **Porównanie** – wszystkie trzy algorytmy działają równolegle na tym samym wejściu; na żywo wyświetlane są makespany i historia na wspólnym wykresie.
3. **Tryb gry** – użytkownik sam gra rolę ogrodnika, klika w bambusy, by je przyciąć, rywalizując z botem Reduce-Max o niższy makespan.

6.3. Kluczowe decyzje implementacyjne

- **Dyskretna symulacja** – każdy krok odpowiada jednemu dniowi: bambusy najpierw rosną, następnie oracle wybiera bambus do przycięcia.
- **Niemutowalny model danych** – funkcje `grow()` i `cut()` zwracają nowe listy bambusów, co upraszcza testowanie i zmniejsza ryzyko przypadkowego nadpisania stanu.
- **Wspólny interfejs strategii** – każda strategia jest funkcją przyjmującą listę bambusów i zwracającą indeks wybranego cięcia.
- **Stan w `st.session_state`** – Streamlit przechowuje aktualny ogród, historię makespanu, wybrany tryb i parametry użytkownika.
- **Renderowanie SVG** – grafika jest stylizowana na ilustrację z artykułu: zielone łodygi bambusa, znaczniki wysokości, linie pomocnicze i robot-panda wskazujący ostatnie cięcie.
- **Testowalna warstwa obliczeniowa** – funkcje symulacyjne są niezależne od Streamlita i są pokryte testami jednostkowymi.

6.4. Uruchomienie i testy

Projekt można uruchomić lokalnie poleceniami:

```
python3.12 -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
streamlit run app.py
```

Do weryfikacji logiki symulacji przygotowano testy jednostkowe:

```
python -m unittest
```

Testy sprawdzają między innymi niezmienniczość operacji `grow()` i `cut()`, wybór bambusa przez Reduce-Max i Reduce-Fastest(x), normalizację współczynników wzrostu oraz poprawne raportowanie makespanu przed cięciem.

6.5. Wkład własny

Warstwa teoretyczna raportu opiera się na wynikach znanych z literatury, przede wszystkim z pracy Bilò i wsp. [1]. Nasz wkład projektowy polega na przełożeniu tych wyników na działające narzędzie eksperymentalne:

- przygotowaliśmy implementację symulacji BGT w Pythonie;
- zaimplementowaliśmy trzy reguły wyboru cięcia i wspólny mechanizm kroku symulacji;
- zbudowaliśmy trzy tryby interakcji: Sandbox, Compare oraz Game;
- dodaliśmy wizualizacje SVG pokazujące aktualne wysokości bambusów, ostatnie cięcie i historię makespanu;
- przygotowaliśmy testy jednostkowe sprawdzające kluczowe operacje modelu.

Dzięki temu raport nie kończy się wyłącznie na omówieniu twierdzeń: pokazuje również, jak abstrakcyjne strategie zachowują się w konkretnych, uruchamialnych instancjach.

7. Analiza złożoności i podsumowanie wyników

7.1. Zestawienie złożoności Oracle’i

Tabela 2: Złożoność czasowa i przestrzenna Trimming Oracle’i.

Strategia / Oracle	Makespan	Budowa	Zapytanie	Przestrzeń
Reduce-Max naiwny	≤ 9	$O(n)$	$O(n)$	$O(n)$
Reduce-Max z otoczką	≤ 9	$O(n \log n)$	$O(\log^2 n)$ lub $O(\log n)$ amort.	$O(n)$
Reduce-Fastest(x)	$\leq 2,62$ dla opt. x	$O(n \log n)$	$O(\log n)$	$O(n)$
Oracle makespan-2	≤ 2	$O(n \log n)$	$O(\log n)$	$O(n)$

Warto podkreślić różnicę między *strategią* a *oracle*. Strategia określa, który bambus powinien zostać przycięty, natomiast oracle odpowiada za efektywne znalezienie tego

bambusa bez przeglądania całego ogrodu. Nasza aplikacja implementuje reguły wyboru bez zaawansowanych struktur danych, ponieważ jej celem jest interaktywna demonstracja i eksperymentowanie na umiarkowanych instancjach, a nie obsługa milionów bambusów.

7.2. Otwarte problemy

Artykuł pozostawia kilka otwartych pytań o dużym znaczeniu:

1. **Hipoteza dla Reduce-Max:** Czy makespan Reduce-Max wynosi co najwyżej 2 dla każdej instancji? Eksperymentalnie odpowiedź jest twierdząca, ale dowód algebraiczny pozostaje nieznany.
2. **Hipoteza dla Reduce-Fastest(1):** Czy Reduce-Fastest(1) gwarantuje makespan 2? To odpowiadałoby optymalności przy zerowym narzucie na selekcję.
3. **Dolne ograniczenie dla Reduce-Fastest(x):** Nie są znane żadne dolne ograniczenia inne niż trywialne ≥ 1 .
4. **Lepsza praktyczna implementacja oracle’i:** Interesujące byłoby porównanie struktur teoretycznych z prostymi implementacjami używającymi kopców, drzew przedziałowych lub struktur bibliotecznych.

8. Wnioski

Problem Bamboo Garden Trimming jest dobrym przykładem sytuacji, w której bardzo prosta reguła działania prowadzi do nietrywialnej analizy kombinatorycznej. Wystarczy jeden robot, jeden ogród i jedno cięcie dziennie, żeby pojawiły się pytania o granice aproksymacji, redukcje do harmonogramowania wieczystego oraz dynamiczne struktury danych.

Główne wyniki artykułu Bilò i wsp. [1] to:

- pierwsze stałe ograniczenie górne na makespan Reduce-Max: ≤ 9 ,
- poprawa ograniczenia dla Reduce-Fastest(x) do $\approx 2,62$ przy optymalnym $x = 1 + 1/\sqrt{5}$,
- trzy wydajne Trimming Oracle’i, w tym Oracle gwarantujący makespan 2 w czasie $O(\log n)$ na zapytanie,
- rozwiązanie otwartego problemu z [2] dotyczącego implementacji Reduce-Max w czasie $o(n)$.

Zaimplementowana przez nas aplikacja webowa pozwala na interaktywne badanie powyższych strategii i wizualną weryfikację ich zachowania na wybranych instancjach. Tryb porównania pokazuje, że różnice między Reduce-Max, Reduce-Fastest(x) i oracle nie są jedynie abstrakcyjne: dla tych samych współczynników wzrostu algorytmy mogą

generować inną historię makespanu i inne decyzje cięcia. Projekt stanowi więc praktyczne uzupełnienie analizy teoretycznej: pozwala zobaczyć, jak twierdzenia przekładają się na dynamikę konkretnego ogrodu.

Literatura

- [1] D. Bilò, L. Gualà, S. Leucci, G. Proietti, G. Scornavacca, *Cutting bamboo down to size*, Theoretical Computer Science **909** (2022), ss. 54–67. <https://doi.org/10.1016/j.tcs.2022.01.027>
- [2] L. Gąsieniec, R. Klasing, C. Levcopoulos, A. Lingas, J. Min, T. Radzik, *Bamboo garden trimming problem (perpetual maintenance of machines with different attendance urgency factors)*, w: SOFSEM 2017, Lecture Notes in Computer Science, Springer 2017, ss. 229–240.
- [3] R. Holte, A. Mok, L. Rosier, I. Tulchinsky, D. Varvel, *The pinwheel: a real-time scheduling problem*, Proceedings of the 22nd Annual Hawaii International Conference on System Sciences, 1989, ss. 693–702.
- [4] M.H.L. Bodlaender, C.A.J. Hurkens, V.J.J. Kusters, F. Staals, G.J. Woeginger, H. Zantema, *Cinderella versus the wicked stepmother*, w: TCS 2012, Lecture Notes in Computer Science, t. 7604, Springer 2012, ss. 57–71.
- [5] M. D’Emidio, G.D. Stefano, A. Navarra, *Bamboo garden trimming problem: priority schedulings*, Algorithms **12**(4) (2019), art. 74. <https://doi.org/10.3390/a12040074>
- [6] S.S. Alshamrani, D.R. Kowalski, L.A. Gąsieniec, *Efficient discovery of malicious symptoms in clouds via monitoring virtual machines*, IEEE CIT/IUCC/DASC/PICom 2015, ss. 1703–1710.
- [7] M.A. Bender, R. Das, M. Farach-Colton, R. Johnson, W. Kuszmaul, *Flushing without cascades*, Proceedings of SODA 2020, ss. 650–669.
- [8] M.H. Goldwasser, *A survey of buffer management policies for packet switches*, SIGACT News **41**(1) (2010), ss. 100–128.
- [9] E.M. McCreight, *Priority search trees*, SIAM Journal on Computing **14**(2) (1985), ss. 257–276.
- [10] M.H. Overmars, J. van Leeuwen, *Maintenance of configurations in the plane*, Journal of Computer and System Sciences **23**(2) (1981), ss. 166–204.
- [11] G.S. Brodal, R. Jacob, *Dynamic planar convex hull*, Proceedings of FOCS 2002, ss. 617–626.
- [12] M.Y. Chan, F.Y.L. Chin, *Schedulers for larger classes of pinwheel instances*, Algorithmica **9**(5) (1993), ss. 425–462.

- [13] M. van Ee, *A $12/7$ -approximation algorithm for the discrete bamboo garden trimming problem*, CoRR, arXiv:2004.11731, 2020.